Case Study 12: Attendance Recovery Computation

Problem, algorithm and standard-symbol flowchart

Problem statement

Find extra attended classes needed to reach 75% attendance.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

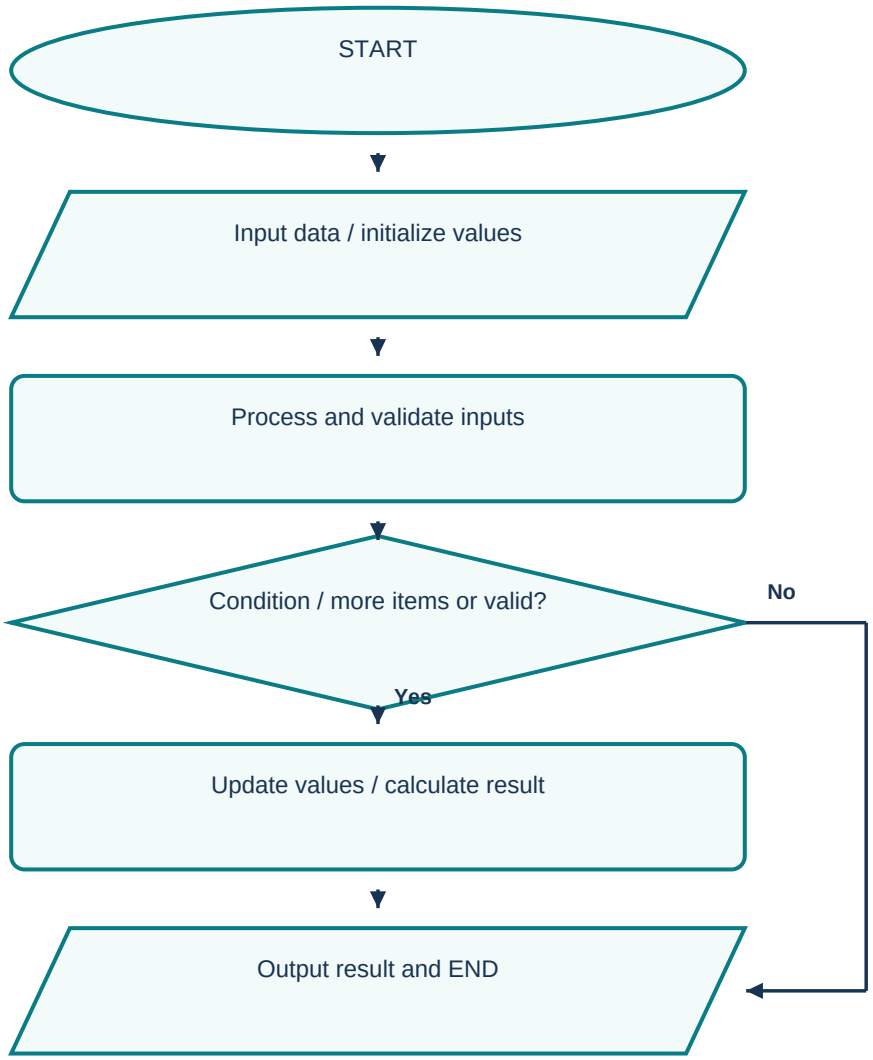
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read total classes and attended classes
2. Reject invalid counts
3. Calculate current attendance
4. While attendance is below 75%, add one attended class
5. Display extra classes and final percentage

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 12: Attendance Recovery Computation

Pseudocode, Python program, test case and conclusion

Pseudocode

READ TOTAL CLASSES AND ATTENDED CLASSES
REJECT INVALID COUNTS
CALCULATE CURRENT ATTENDANCE
WHILE ATTENDANCE IS BELOW 75%, ADD ONE ATTENDED CLASS
DISPLAY EXTRA CLASSES AND FINAL PERCENTAGE

Python source code

```
total = int(input("Total classes: "))
attended = int(input("Attended classes: "))
if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance")
else:
    extra = 0
    while attended * 100 < 75 * total:
        total += 1
        attended += 1
        extra += 1
    print("Extra classes:", extra)
    print("Attendance:", round(attended / total * 100, 2))
```

Test case and expected sample output

Total classes: 100; Attended classes: 70 -> Extra classes: 20; Attendance: 75.0

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 13: ATM Transaction Sequence Simulation

Problem, algorithm and standard-symbol flowchart

Problem statement

Simulate deposits and withdrawals with balance and withdrawal checks.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

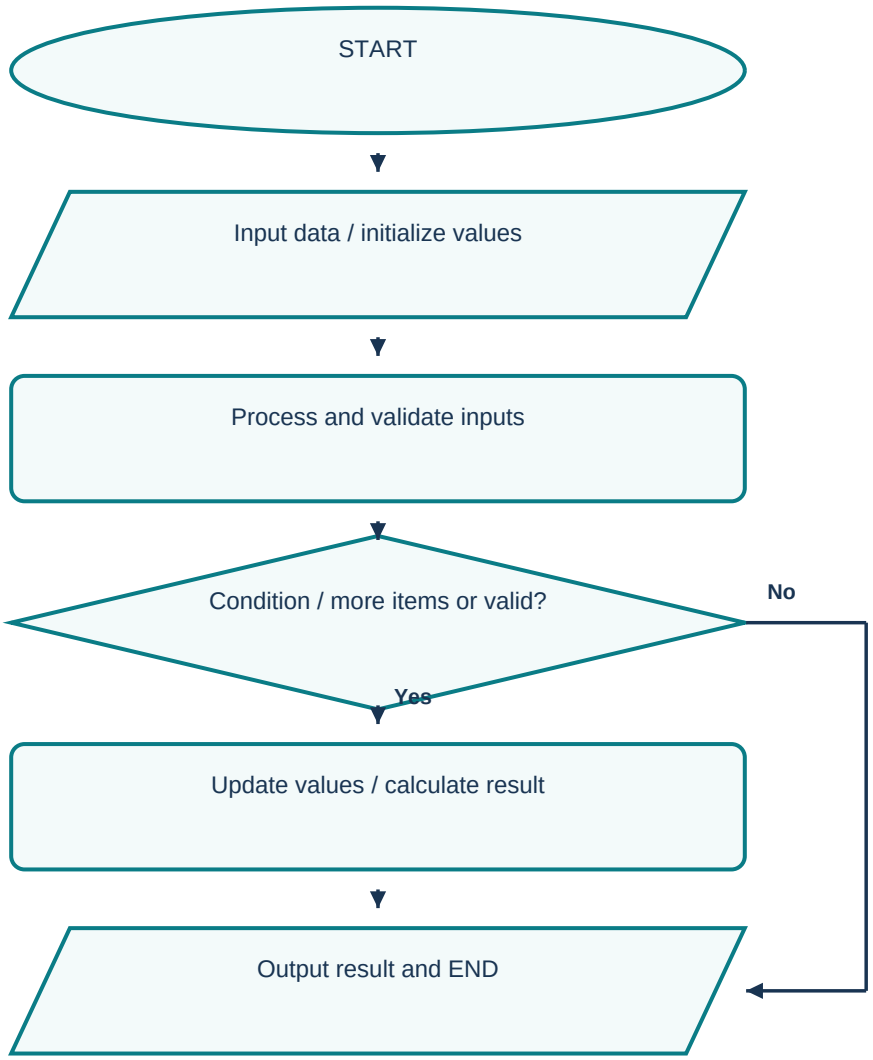
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Initialize balance to 5000
2. Read number of transactions
3. For each transaction, read choice and amount
4. Validate and update balance for valid deposits/withdrawals
5. Display final balance and successful count

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 13: ATM Transaction Sequence Simulation

Pseudocode, Python program, test case and conclusion

Pseudocode

INITIALIZE BALANCE TO 5000
READ NUMBER OF TRANSACTIONS
FOR EACH TRANSACTION, READ CHOICE AND AMOUNT
VALIDATE AND UPDATE BALANCE FOR VALID DEPOSITS/WITHDRAWALS
DISPLAY FINAL BALANCE AND SUCCESSFUL COUNT

Python source code

```
balance = 5000
successful = 0
n = int(input("Transactions: "))
for i in range(n):
    choice = int(input("1 Deposit, 2 Withdraw: "))
    amount = int(input("Amount: "))
    if amount <= 0:
        print("Invalid amount")
    elif choice == 1:
        balance += amount
        successful += 1
        print("Deposited")
    elif choice == 2:
        if amount > 10000:
            print("Withdrawal limit exceeded")
        elif amount > balance:
            print("Insufficient balance")
        else:
            balance -= amount
            successful += 1
            print("Withdrawn")
    else:
        print("Invalid choice")
print("Final balance:", balance)
print("Successful transactions:", successful)
```

Test case and expected sample output

Transactions: 2; deposit 1000; withdraw 500 -> Final balance: 5500; Successful transactions: 2

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 19: Multi-Product Shopping Budget Constraint

Problem, algorithm and standard-symbol flowchart

Problem statement

Calculate a basket total, apply a sample discount, and compare with budget.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

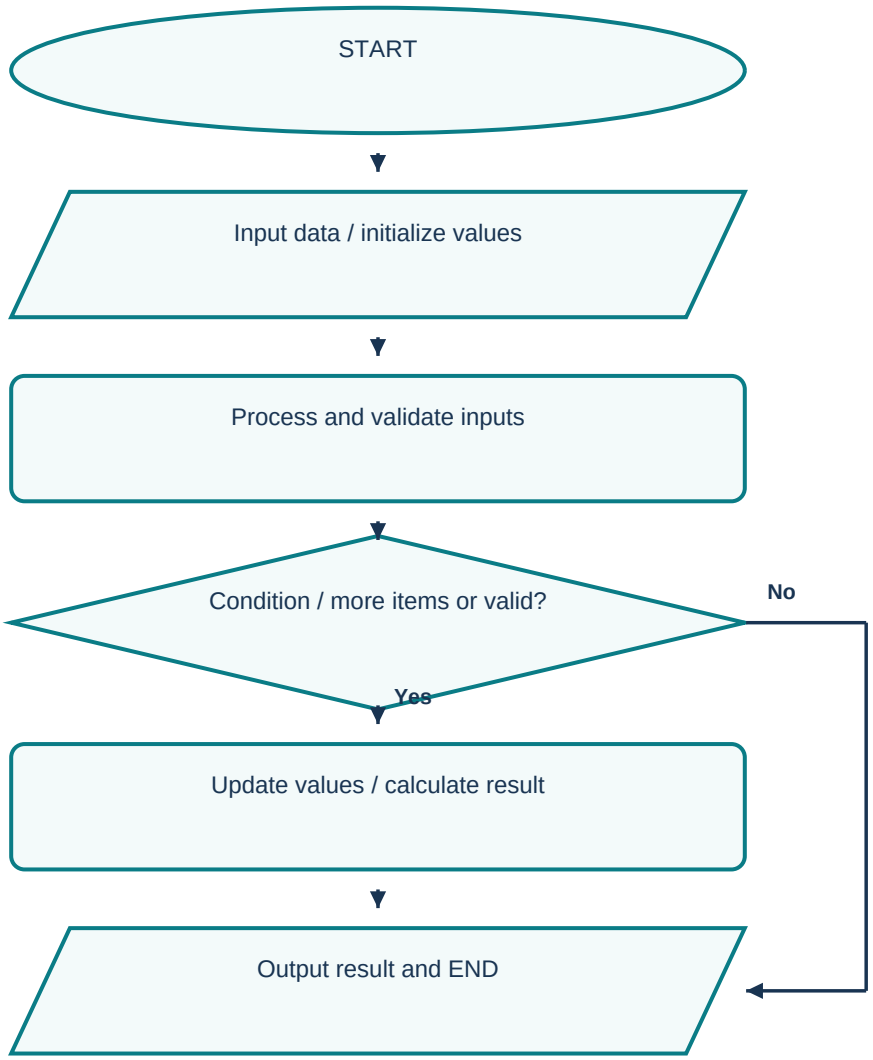
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read budget and number of products
2. Read each product price and quantity
3. Add valid item costs to total
4. Apply 10% discount when subtotal exceeds 1000
5. Compare total with budget and display difference

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 19: Multi-Product Shopping Budget Constraint

Pseudocode, Python program, test case and conclusion

Pseudocode

READ BUDGET AND NUMBER OF PRODUCTS
READ EACH PRODUCT PRICE AND QUANTITY
ADD VALID ITEM COSTS TO TOTAL
APPLY 10% DISCOUNT WHEN SUBTOTAL EXCEEDS 1000
COMPARE TOTAL WITH BUDGET AND DISPLAY DIFFERENCE

Python source code

```
budget = float(input("Budget: "))
n = int(input("Products: "))
total = 0
for i in range(n):
    name = input("Product: ")
    price = float(input("Price: "))
    qty = int(input("Quantity: "))
    if price < 0 or qty < 0:
        print("Invalid item")
    else:
        total += price * qty
if total > 1000:
    total *= 0.90
print("Bill:", round(total, 2))
if total <= budget:
    print("Within budget")
else:
    print("Excess:", round(total - budget, 2))
```

Test case and expected sample output

Budget: 1000; one item price 1200, quantity 1 -> Bill: 1080.0; Excess: 80.0

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 29: Password Policy Validation Engine

Problem, algorithm and standard-symbol flowchart

Problem statement

Check password length, character types, repetitions, and banned words.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

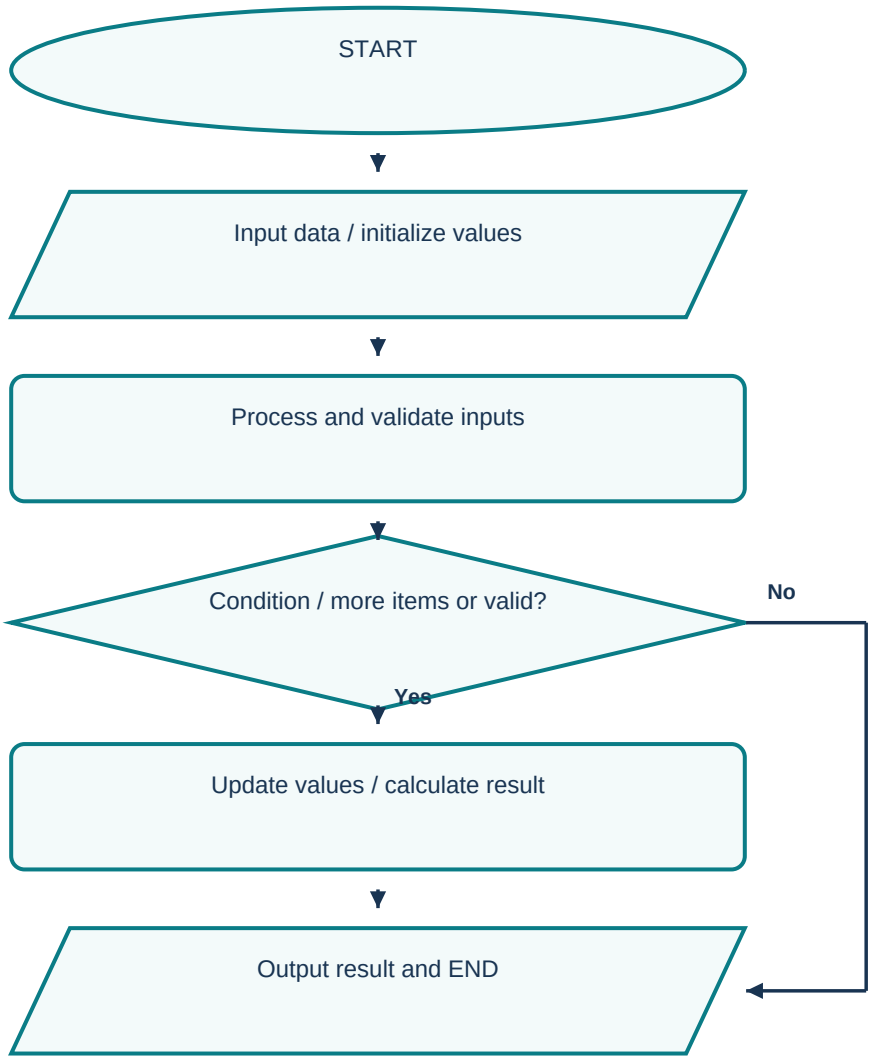
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read password
2. Scan characters for uppercase, lowercase, digit, special
3. Check adjacent repetition
4. Check minimum length and banned substring
5. Display validity and reasons

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 29: Password Policy Validation Engine

Pseudocode, Python program, test case and conclusion

Pseudocode

READ PASSWORD
SCAN CHARACTERS FOR UPPERCASE, LOWERCASE, DIGIT, SPECIAL
CHECK ADJACENT REPETITION
CHECK MINIMUM LENGTH AND BANNED SUBSTRING
DISPLAY VALIDITY AND REASONS

Python source code

```
password = input("Password: ")
upper = lower = digit = special = repeated = False
for i in range(len(password)):
    ch = password[i]
    if ch.isupper(): upper = True
    elif ch.islower(): lower = True
    elif ch.isdigit(): digit = True
    else: special = True
    if i > 0 and ch == password[i-1]:
        repeated = True
errors = []
if len(password) < 8: errors.append("Too short")
if not upper: errors.append("Uppercase missing")
if not lower: errors.append("Lowercase missing")
if not digit: errors.append("Digit missing")
if not special: errors.append("Special character missing")
if repeated: errors.append("Adjacent repetition")
if "password" in password.lower():
    errors.append("Banned word")
if errors:
    for error in errors: print(error)
    print("Invalid password")
else:
    print("Valid password")
```

Test case and expected sample output

Password: Abc123!x -> Valid password

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 31: String-Based Examination Answer Evaluation

Problem, algorithm and standard-symbol flowchart

Problem statement

Score an answer string with +4 for correct, -1 for wrong, 0 for blank.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

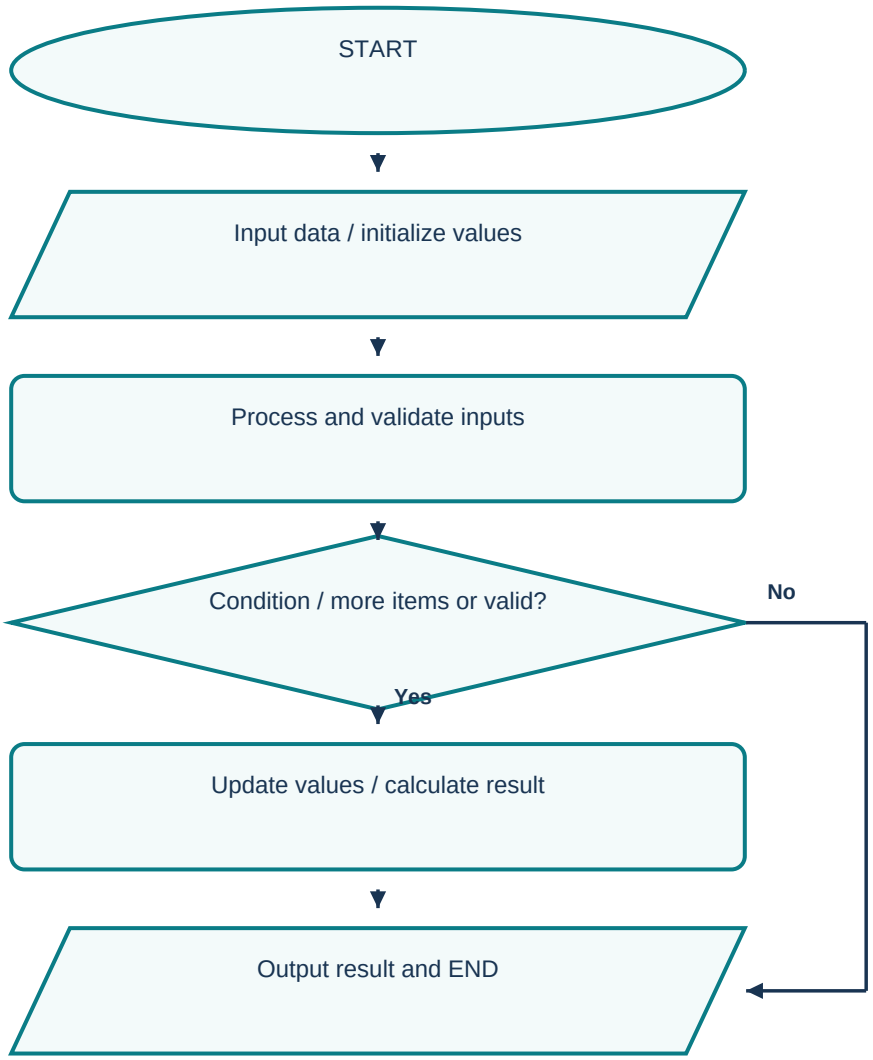
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Store answer key
2. Read student answers
3. Check answer-string length
4. Count correct, wrong, and unanswered responses
5. Calculate and display marks

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 31: String-Based Examination Answer Evaluation

Pseudocode, Python program, test case and conclusion

Pseudocode

STORE ANSWER KEY
READ STUDENT ANSWERS
CHECK ANSWER-STRING LENGTH
COUNT CORRECT, WRONG, AND UNANSWERED RESPONSES
CALCULATE AND DISPLAY MARKS

Python source code

```
key = "ABCDABCD"
answers = input("Answers (X for blank): ").upper()
if len(answers) != len(key):
    print("Invalid answer count")
else:
    correct = wrong = blank = 0
    for i in range(len(key)):
        if answers[i] == "X": blank += 1
        elif answers[i] == key[i]: correct += 1
        else: wrong += 1
    score = 4 * correct - wrong
    print("Correct:", correct)
    print("Wrong:", wrong)
    print("Blank:", blank)
    print("Score:", score)
```

Test case and expected sample output

Answers: ABCDABCX -> Correct: 7; Wrong: 0; Blank: 1; Score: 28

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 35: Text Compression Using Character Runs

Problem, algorithm and standard-symbol flowchart

Problem statement

Encode consecutive character runs and compare lengths.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

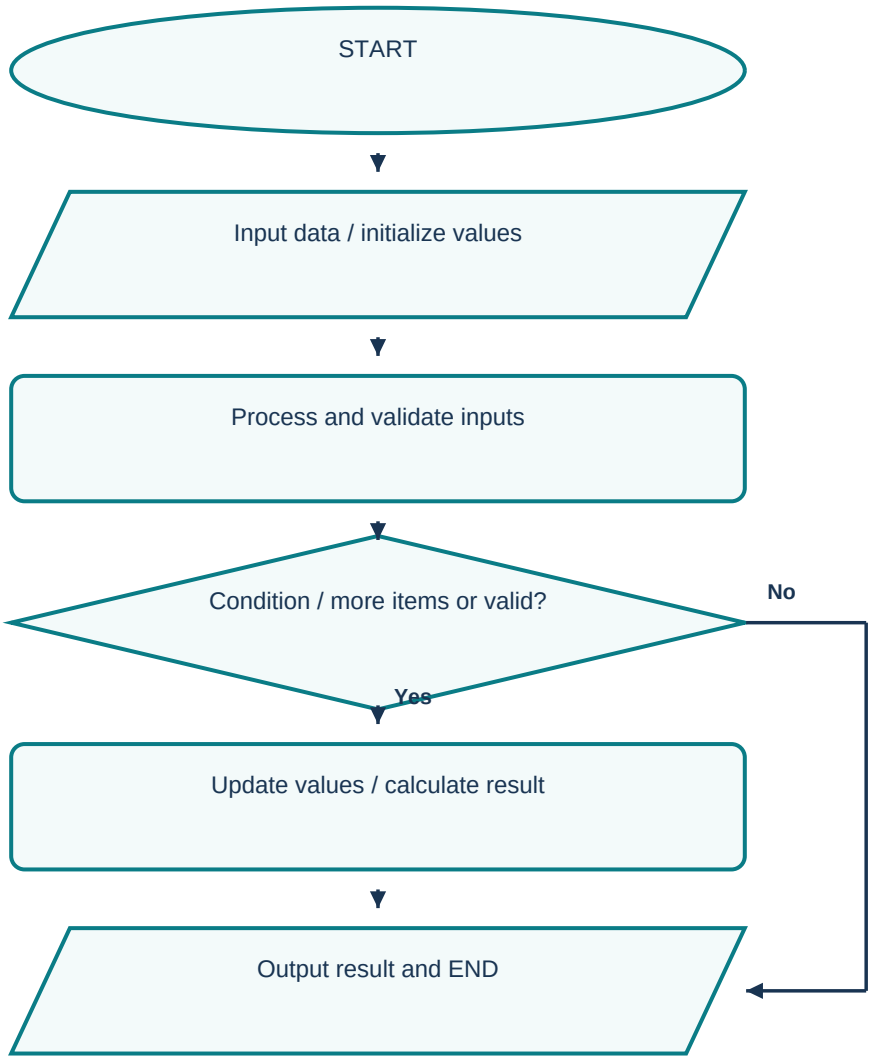
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read string
2. Start count at 1
3. Scan adjacent characters and count repeats
4. Append character followed by run length
5. Compare original and encoded lengths

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 35: Text Compression Using Character Runs

Pseudocode, Python program, test case and conclusion

Pseudocode

READ STRING
START COUNT AT 1
SCAN ADJACENT CHARACTERS AND COUNT REPEATS
APPEND CHARACTER FOLLOWED BY RUN LENGTH
COMPARE ORIGINAL AND ENCODED LENGTHS

Python source code

```
text = input("Text: ")
encoded = ""
count = 1
for i in range(1, len(text)):
    if text[i] == text[i-1]:
        count += 1
    else:
        encoded += text[i-1] + str(count)
        count = 1
if text:
    encoded += text[-1] + str(count)
print("Encoded:", encoded)
if len(encoded) < len(text):
    print("Compression helps")
else:
    print("Compression does not help")
```

Test case and expected sample output

Text: aaabbbb -> Encoded: a3b4; Compression helps

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 36: Repeated Word and Phrase Detection

Problem, algorithm and standard-symbol flowchart

Problem statement

Find repeated words and their positions after basic normalization.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

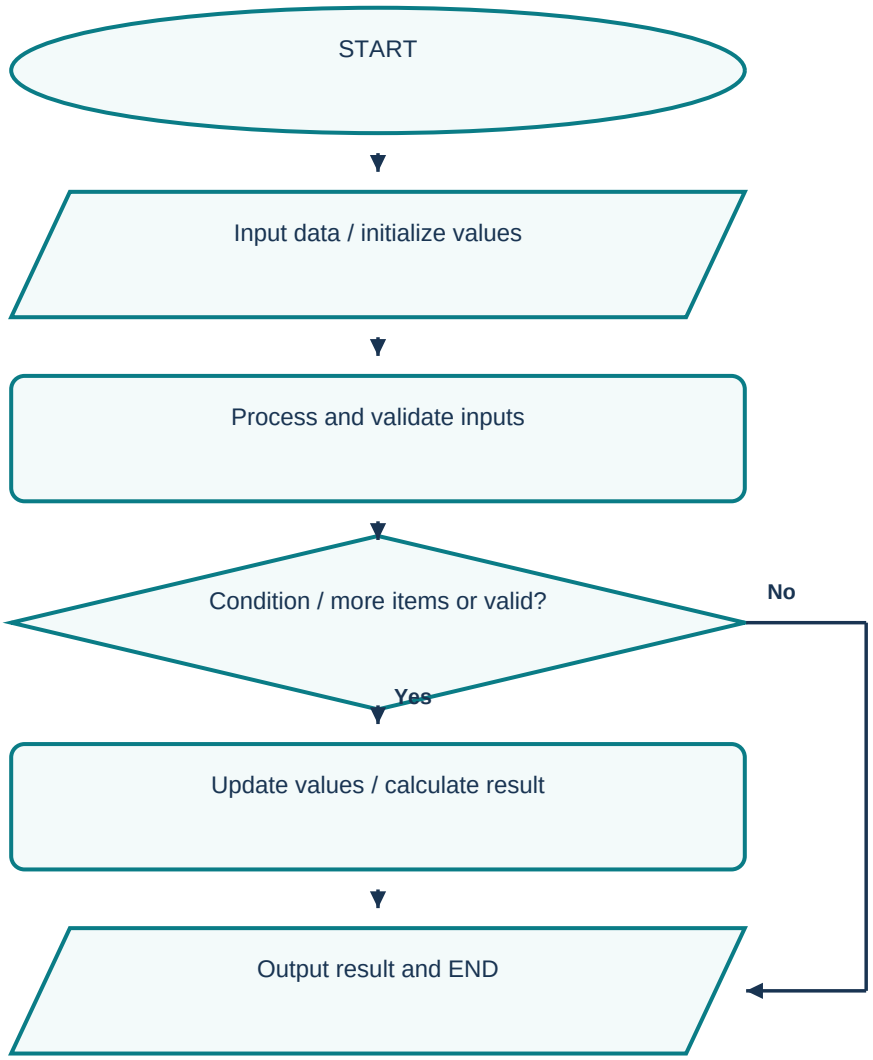
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read sentence and convert to lowercase
2. Split sentence into words
3. Remove common edge punctuation
4. For each distinct word, find its positions
5. Print words occurring more than once

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 36: Repeated Word and Phrase Detection

Pseudocode, Python program, test case and conclusion

Pseudocode

READ SENTENCE AND CONVERT TO LOWERCASE
SPLIT SENTENCE INTO WORDS
REMOVE COMMON EDGE PUNCTUATION
FOR EACH DISTINCT WORD, FIND ITS POSITIONS
PRINT WORDS OCCURRING MORE THAN ONCE

Python source code

```
sentence = input("Sentence: ").lower()
words = []
for item in sentence.split():
    words.append(item.strip(".,!?;:"))
seen = []
found = False
for word in words:
    if word not in seen:
        seen.append(word)
        positions = []
        for i in range(len(words)):
            if words[i] == word:
                positions.append(i + 1)
        if len(positions) > 1:
            print(word, len(positions), positions)
            found = True
if not found:
    print("No repeated words")
```

Test case and expected sample output

Sentence: Cat dog cat -> cat 2 [1, 3]

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 51: Bus Passenger Occupancy Simulation

Problem, algorithm and standard-symbol flowchart

Problem statement

Track boarding, alighting, and capacity at each stop.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

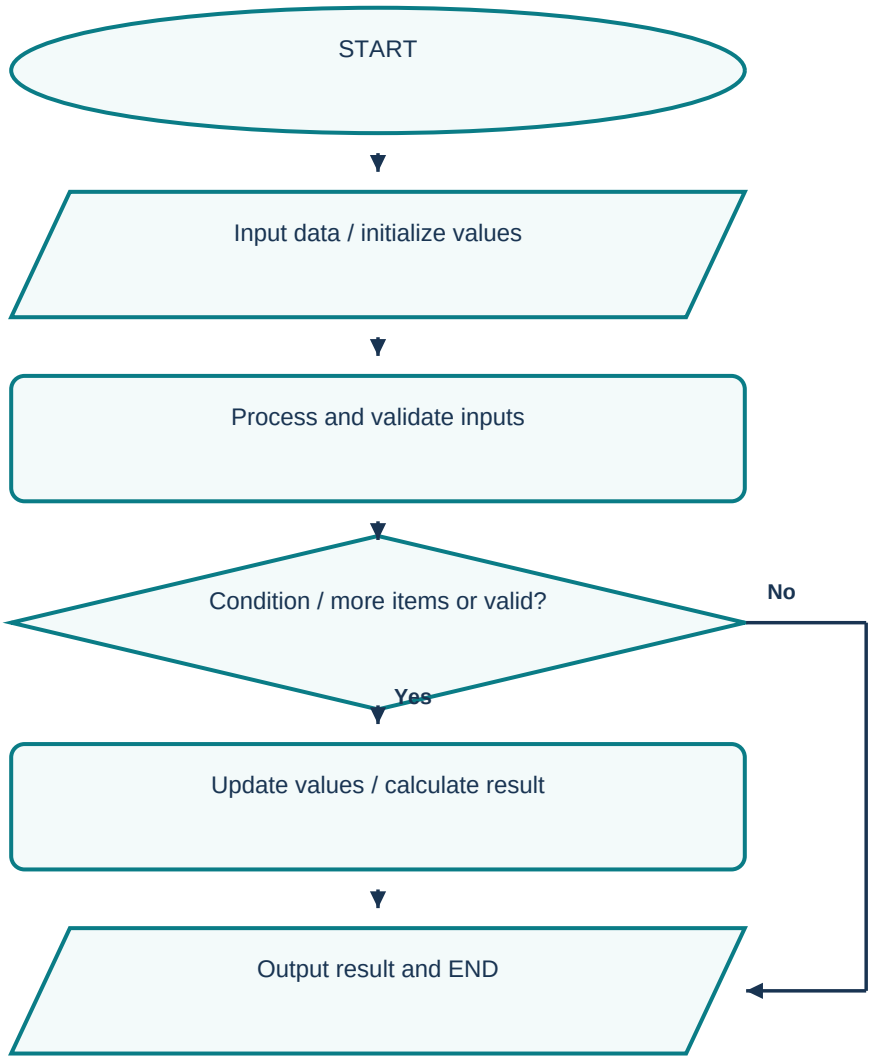
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read capacity and stop count
2. Initialize passengers to zero
3. For each stop, read boarding and leaving
4. Reject invalid counts, otherwise update passengers
5. Warn when occupancy exceeds capacity

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 51: Bus Passenger Occupancy Simulation

Pseudocode, Python program, test case and conclusion

Pseudocode

READ CAPACITY AND STOP COUNT
INITIALIZE PASSENGERS TO ZERO
FOR EACH STOP, READ BOARDING AND LEAVING
REJECT INVALID COUNTS, OTHERWISE UPDATE PASSENGERS
WARN WHEN OCCUPANCY EXCEEDS CAPACITY

Python source code

```
capacity = int(input("Capacity: "))
stops = int(input("Stops: "))
passengers = 0
for i in range(stops):
    print("Stop", i + 1)
    boarding = int(input("Boarding: "))
    leaving = int(input("Leaving: "))
    if boarding < 0 or leaving < 0 or leaving > passengers:
        print("Invalid counts")
    else:
        passengers += boarding - leaving
        print("Passengers:", passengers)
        if passengers > capacity:
            print("Over capacity")
print("Final passengers:", passengers)
```

Test case and expected sample output

Capacity: 20; Stops: 1; Boarding: 10; Leaving: 0 -> Passengers: 10

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 53: Food Waste Monitoring System

Problem, algorithm and standard-symbol flowchart

Problem statement

Compute daily leftover meals and flag wastage above 10%.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

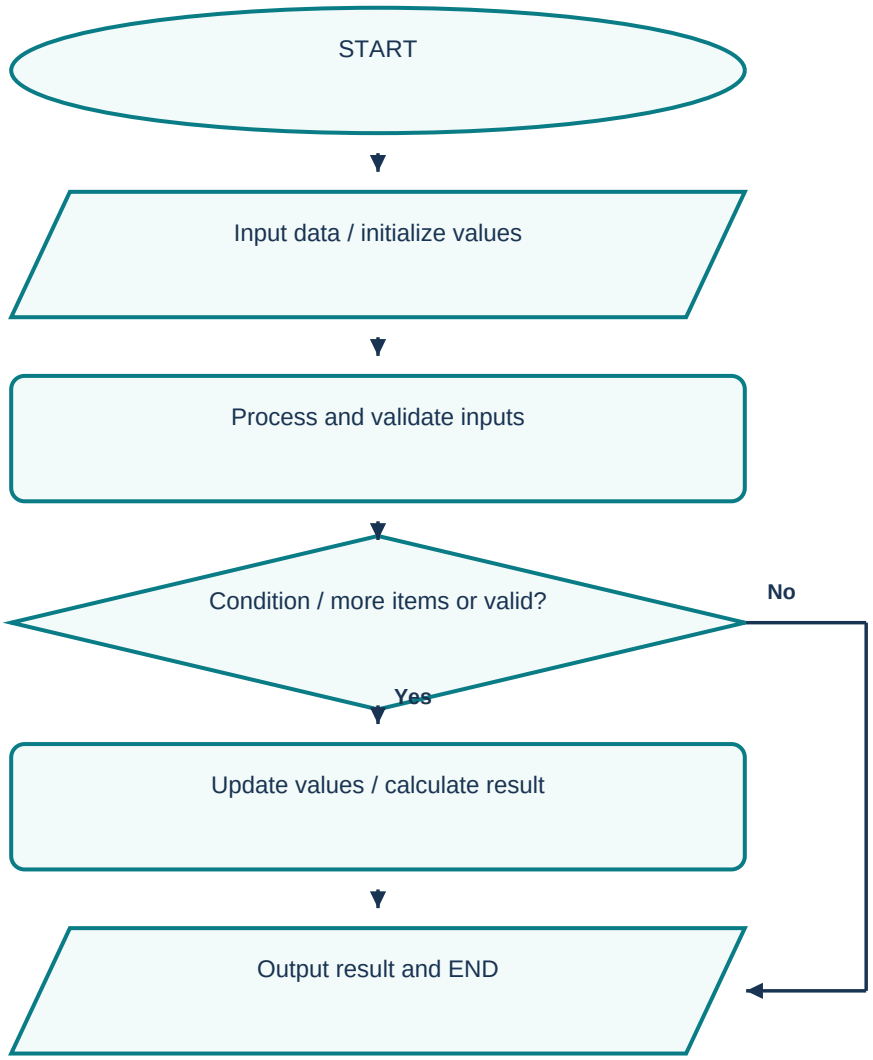
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read number of days
2. For each day, read prepared and served meals
3. Validate quantities
4. Compute waste and waste percentage
5. Flag days exceeding 10% and display total

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 53: Food Waste Monitoring System

Pseudocode, Python program, test case and conclusion

Pseudocode

READ NUMBER OF DAYS
FOR EACH DAY, READ PREPARED AND SERVED MEALS
VALIDATE QUANTITIES
COMPUTE WASTE AND WASTE PERCENTAGE
FLAG DAYS EXCEEDING 10% AND DISPLAY TOTAL

Python source code

```
days = int(input("Days: "))
total_waste = 0
for i in range(days):
    prepared = int(input("Prepared: "))
    served = int(input("Served: "))
    if prepared <= 0 or served < 0 or served > prepared:
        print("Invalid quantities")
        continue
    waste = prepared - served
    percent = waste / prepared * 100
    total_waste += waste
    print("Waste:", waste, "Percent:", round(percent, 2))
    if percent > 10:
        print("High wastage")
print("Total waste:", total_waste)
```

Test case and expected sample output

Days: 1; Prepared: 100; Served: 85 -> Waste: 15; Percent: 15.0; High wastage

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.

Case Study 63: Multi-Product Price Fluctuation Analysis

Problem, algorithm and standard-symbol flowchart

Problem statement

Calculate percentage price changes and identify the largest fluctuation.

Objective

Build a simple Python solution using only syllabus-level variables, conditions, loops, strings and lists.

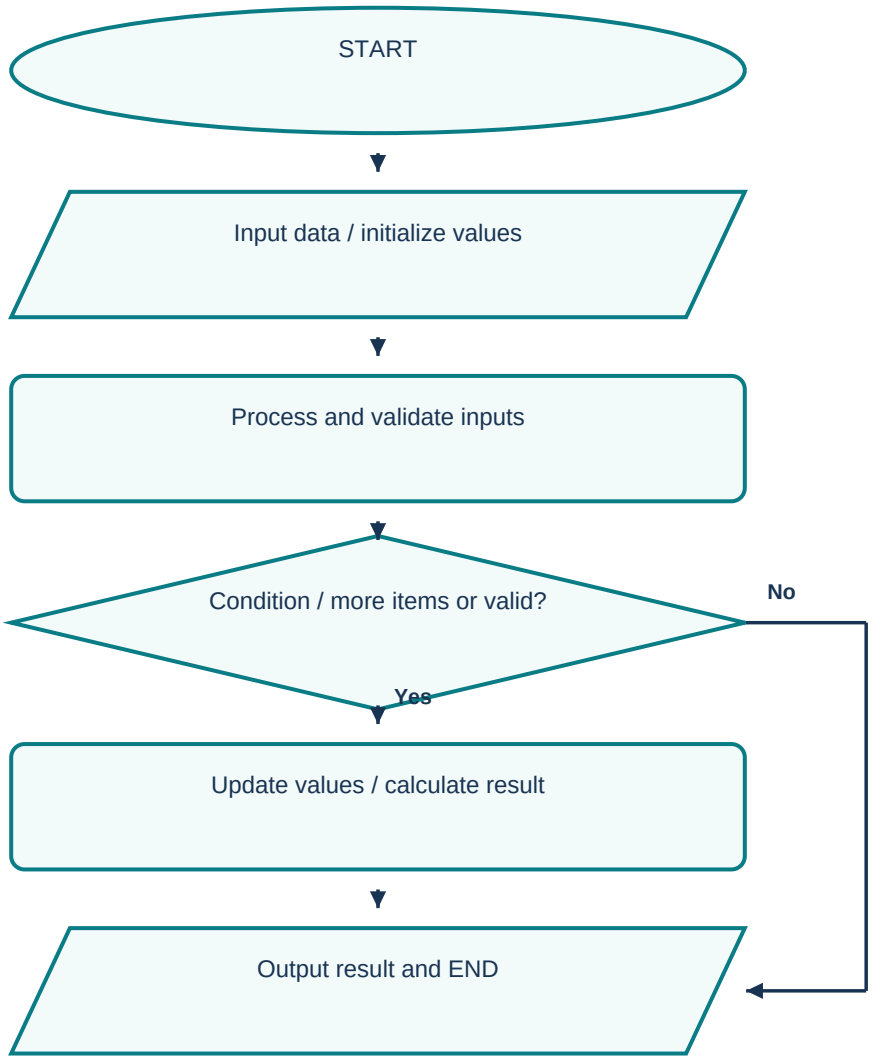
Assumptions

Rules and thresholds are illustrative because the question sheet does not provide exact policies.

Algorithm

1. Read number of products
2. For each product, read name and old/new price
3. Reject invalid prices
4. Calculate percentage change and flag absolute change over 10%
5. Track and display largest absolute fluctuation

Flowchart (standard shapes)



Oval: Start/End Parallelogram: Input/Output Rectangle: Process Diamond: Decision

Case Study 63: Multi-Product Price Fluctuation Analysis

Pseudocode, Python program, test case and conclusion

Pseudocode

READ NUMBER OF PRODUCTS
FOR EACH PRODUCT, READ NAME AND OLD/NEW PRICE
REJECT INVALID PRICES
CALCULATE PERCENTAGE CHANGE AND FLAG ABSOLUTE CHANGE OVER 10%
TRACK AND DISPLAY LARGEST ABSOLUTE FLUCTUATION

Python source code

```
n = int(input("Products: "))
maximum = -1
largest = "None"
for i in range(n):
    name = input("Product: ")
    old = float(input("Old price: "))
    new = float(input("New price: "))
    if old <= 0 or new < 0:
        print("Invalid prices")
        continue
    change = (new - old) / old * 100
    print("Change:", round(change, 2), "%")
    if abs(change) > 10:
        print("Significant fluctuation")
    if abs(change) > maximum:
        maximum = abs(change)
        largest = name
print("Largest fluctuation:", largest)
if maximum >= 0:
    print("Magnitude:", round(maximum, 2), "%")
```

Test case and expected sample output

Products: 1; Product: Pen; Old: 10; New: 12 -> Change: 20.0%; Largest fluctuation: Pen

Conclusion

The program demonstrates the case-study logic using beginner-level Python and displays the calculated result.

Note: Run the code and capture actual screenshots for the required submission.